

Your App Is Up. Your Business Is Down.

Eleftheria Batsou

Developer Advocate, UptimeRobot





Your monitoring is green.



Your monitoring is green.

**And your customers can't pay
you.**



UptimeRobot

External uptime monitoring for websites, APIs and services.

3.4M+

people watching their services from the outside

Before this: a developer. Both sides of the 2am alert.



The code you wrote

- Watched by every check you have



What your code depends on

- Where most outages actually happen

Most monitoring watches the first. It misses the second.

The twenty minutes nobody noticed



THURSDAY, 1:00 PM

● Monitors		1:00 pm
● Homepage	200	184 ms
● API	200	96 ms
● Login page	200	211 ms

Nothing is wrong.

THURSDAY, 2:00 PM

● Monitors		2:00 pm
● Homepage	200	184 ms
● API	200	96 ms
● Login page	200	211 ms



"The checkout button doesn't do anything."

Support ticket #4812 · 2:00 pm

Probably their browser.

Still green.

THURSDAY, 2:30 PM

● Monitors		2:30 pm
● Homepage	200	184 ms
● API	200	96 ms
● Login page	200	211 ms

Still green.



"Checkout button does nothing."

Ticket #4812 · 2:00 pm



"Can't complete my order."

Ticket #4817 · 2:20 pm



"Payment just spins."

Ticket #4819 · 2:20 pm



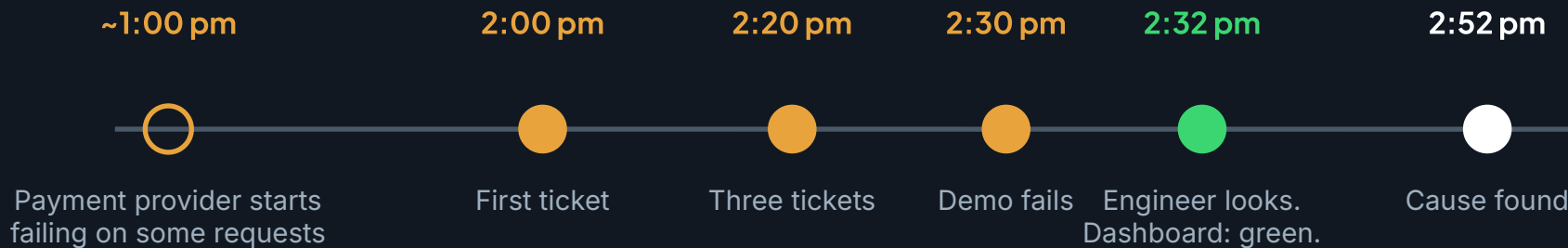
Sales: "Demo just failed in front of a prospect."

● Monitors		2:32 pm
● Homepage	200	184 ms
● API	200	96 ms
● Login page	200	211 ms



Every single check is passing.

WHAT ACTUALLY HAPPENED



Nothing you built was broken.

Your app was up for the entire incident.

~40 min

of revenue

1

demo

some

trust

SWAP IN ANY DEPENDENCY



**Payment
provider**

Nobody can pay



Identity provider

Nobody can log in



CDN

Half your assets
don't load



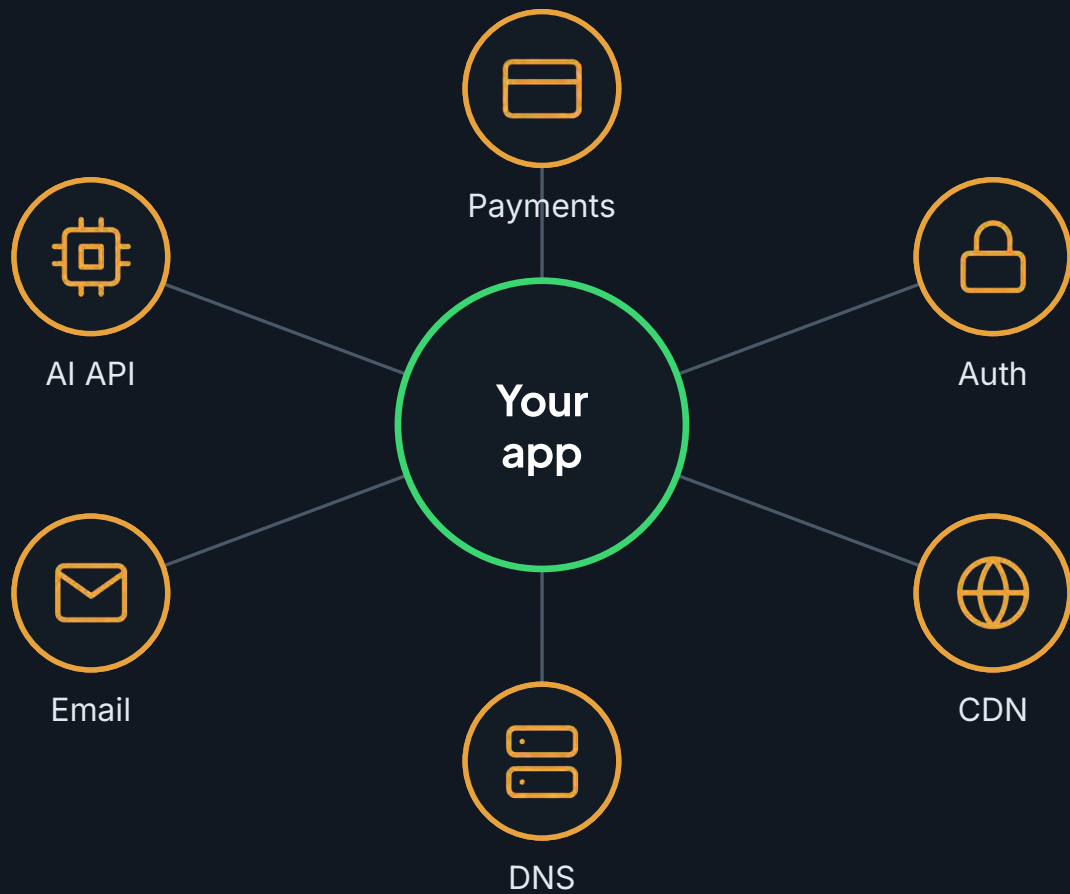
LLM API

Your headline feature
returns nothing

Same story. Different name.

**The thing that broke
isn't the thing you built**

THE SHAPE OF A MODERN APPLICATION



Your code

- ✓ Read it
- ✓ Test it
- ✓ Deploy it
- ✓ Roll it back

A dependency

- 👁️ Observe it from the outside
- ✗ Can't read it
- ✗ Can't deploy it
- ✗ Can't roll it back

You can't fix Stripe. You can only find out fast.

A dependency failure looks like nothing, until it looks like everything.



A few percent of
requests failing



Latency: 200 ms → 4 s



Valid response. Empty
body.

The process is running. The front door opens. Your app is “up.”

● Monitors

now

● Homepage

200

184 ms

● API

200

96 ms

● Login page

200

211 ms

It's measuring the thing you built.

The thing you built is fine.

Why adding more checks doesn't fix it

JUST ADD MORE MONITORS?

Before



After



Ten green lights instead of three.

More confidence. Same coverage. Now you trust the dashboard more.

THREE WAYS A HEALTHY CHECK MISSES A REAL FAILURE

1

The 200 that lies

2

The partial failure

3

The slow success

200

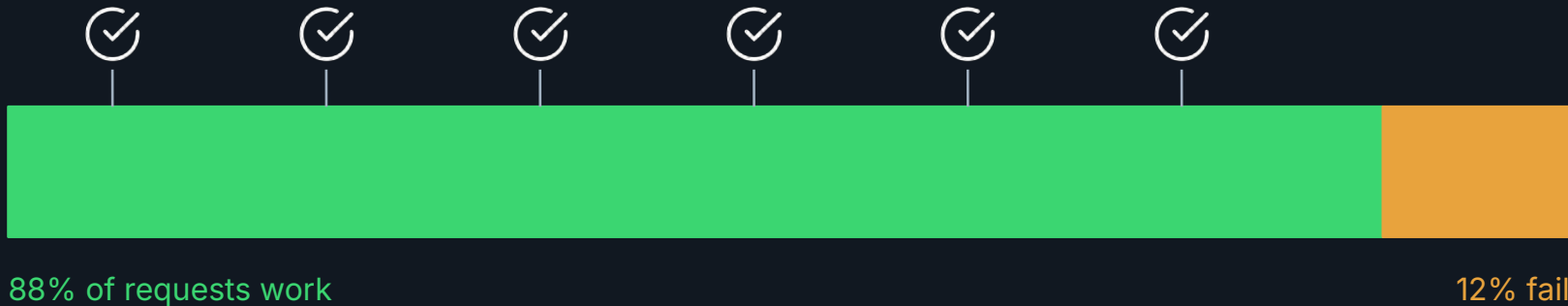
The page rendered.

The payment didn't.

A status code tells you the server answered. Not that the answer was right.

2 · THE PARTIAL FAILURE

Every 5 minutes: check



Six checks. Six passes.

A meaningful slice of your customers can't buy anything.

3 · THE SLOW SUCCESS



Everything completes. Eventually.

Check what the customer does, not what the server says.

Plain HTTP check

"Did the server respond?"

Status code. Nothing else.

Keyword or API check

*"Can a customer still do
the thing they came for?"*

Expected content. Real response.
Human-scale latency.

Same URL. Completely different question.

**Most teams don't actually know
what they depend on**

WHAT DOES YOUR APP DEPEND ON?



Payments



Auth



Database

“That’s everything, right?”

WHAT DOES YOUR APP DEPEND ON?



Payments



Auth



Database



Email
provider



Analytics
script



Font CDN



Map tiles



Feature
flags



Error tracker



That AI API from 8 months ago

Nobody has this list from memory.

WHERE DEPENDENCIES HIDE



Script tags and
third-party includes



Outbound calls
in your backend



DNS records,
especially CNAMEs



TLS certificates
issued to a vendor



Webhooks
you receive



Env vars full of
unaudited API keys

Two hours of unglamorous work. Highest-value monitoring task most teams never do.

GROUP THEM

Payments

Auth

CDN & DNS

Email

Analytics

AI

For each group, two questions:

1

If this degrades, what does a customer experience?

2

Would we know?

Honest answer, usually: "we'd find out from a support ticket."



Font CDN

Don't need to



Payments

Absolutely need to

Monitor the thing that takes money.

**You can't trust a platform
to report its own outage**

"IF STRIPE IS DOWN, STRIPE WILL TELL US."

● All Systems Operational

status.vendor.com

API

Operational

Dashboard

Operational

Checkout

Operational

Webhooks

Operational

Useful. Run in good faith, mostly. And not enough.

**The party with the incentive
to under-report
is the party doing the reporting.**

Structural, not moral.

SO STATUS PAGES...



Lag

"Degraded performance in some regions" while your checkout is dead.



Resolve early

Marked resolved before the tail of the failure has actually cleared.



Miss the partial

The degradation that hurts you most is "within tolerance" on their side.

WE HAVE DATA ON THIS

89%

**of user-reported incidents had no official
status page filing within six hours**

H1 2026 · only services that publish a status page · UptimeRobot data

AND WHEN THE VENDOR DID FILE...



The crowd reports it



The vendor files it



~2 hours

median



**Two hours is a long time
when it's your checkout.**

EVEN WHEN EVERYONE IS HONEST



The vendor's experience of the incident

That's what the status page reports.

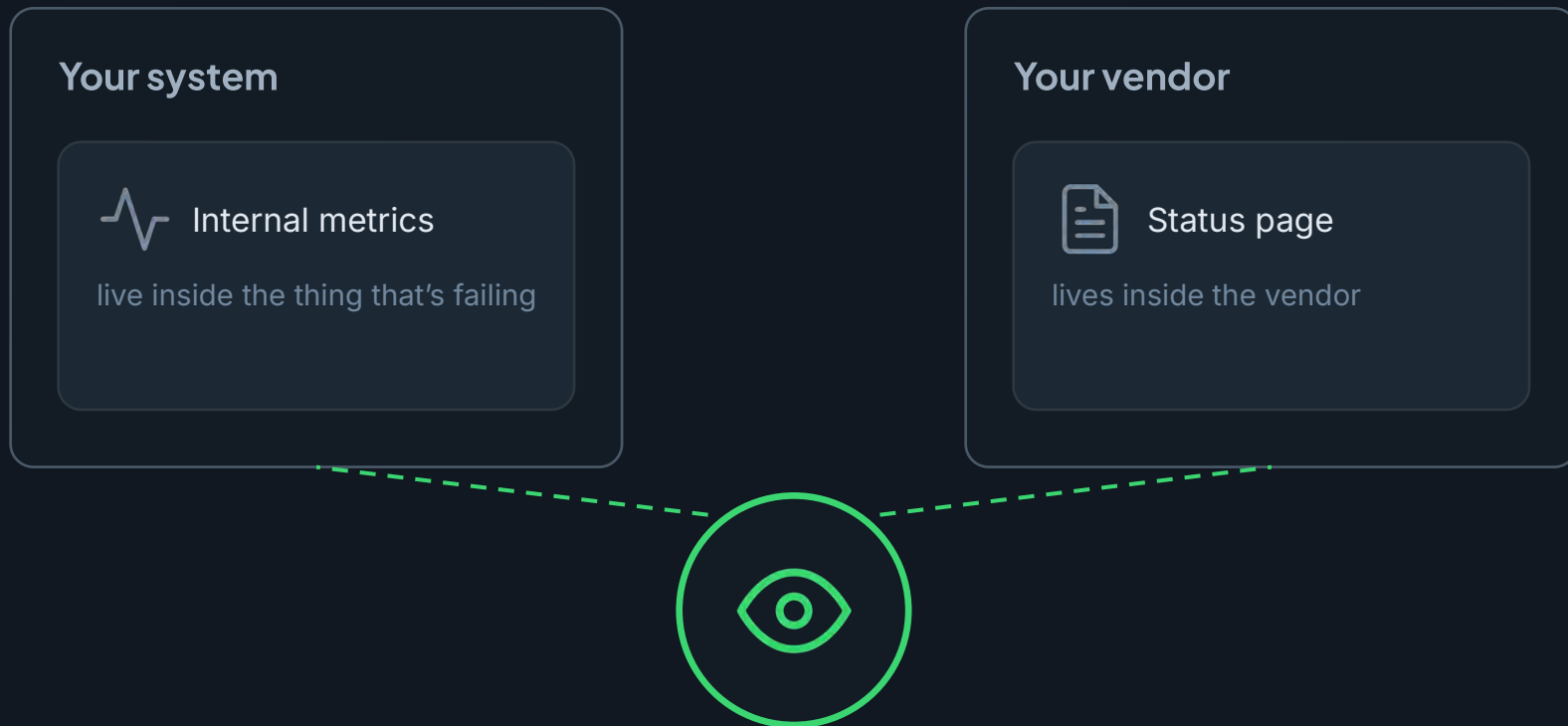


Your experience of the incident

That's what you need to know.

Those are different things.

THE OUTSIDE OBSERVER



Outside both. Same requests a customer makes. From where customers are.

A dashboard is a view from the inside.

You need at least one view from the outside.



What to do this week

THIS WEEK

1

Build the dependency list

Homepage, key pages, backend calls, DNS, API keys.
Two hours. Write it down.

2

Check the dependency itself

3

Upgrade “did it return 200”

4

Put the checks outside

5

Believe your checks

THIS WEEK

1

Build the dependency list

Homepage, key pages, backend calls, DNS, API keys.
Two hours. Write it down.

2

Check the dependency itself

An external check that calls each critical dependency
the way your app does.

3

Upgrade “did it return 200”

4

Put the checks outside

5

Believe your checks

THIS WEEK

1

Build the dependency list

Homepage, key pages, backend calls, DNS, API keys.
Two hours. Write it down.

2

Check the dependency itself

An external check that calls each critical dependency
the way your app does.

3

Upgrade “did it return 200”

To “did it return what a customer needs.” Keywords,
response validation, human-scale latency.

4

Put the checks outside

5

Believe your checks

THIS WEEK

1

Build the dependency list

Homepage, key pages, backend calls, DNS, API keys. Two hours. Write it down.

2

Check the dependency itself

An external check that calls each critical dependency the way your app does.

3

Upgrade “did it return 200”

To “did it return what a customer needs.” Keywords, response validation, human-scale latency.

4

Put the checks outside

Outside your infrastructure. Outside your vendors. Independence is the point.

5

Believe your checks

THIS WEEK

1

Build the dependency list

Homepage, key pages, backend calls, DNS, API keys. Two hours. Write it down.

2

Check the dependency itself

An external check that calls each critical dependency the way your app does.

3

Upgrade “did it return 200”

To “did it return what a customer needs.” Keywords, response validation, human-scale latency.

4

Put the checks outside

Outside your infrastructure. Outside your vendors. Independence is the point.

5

Believe your checks

When the status page says all clear and your checks disagree, your checks are right.

App up $\neq$ Business up

Modern architecture pulled them apart faster than most monitoring caught up.



Try it on the free plan

50 monitors · no credit card

Enough to cover the dependency list you're about to build.



Eleftheria Batsou

Developer Advocate, UptimeRobot

@BatsouElef · uptimerobot.com

*Got a green-dashboard war story?
I'd love to hear it.*